

INF672 / UNIDAD 2 / 2.4 / GL08

GUÍA DE LABORATORIO

Selección y extracción de características

No todas las columnas ayudan: unas son irrelevantes, otras redundantes, y demasiadas a la vez perjudican al modelo. En este laboratorio practicamos cómo quedarnos con lo que importa —seleccionando— o condensar la información en menos dimensiones —extrayendo con PCA—.

MATERIA	DOCENTE	MODALIDAD	DURACIÓN
Machine Learning	M. Sc. Huáscar F. Gonzales G.	Laboratorio guiado	1 sesión de laboratorio

IDEA CLAVE DEL LABORATORIO

Reducir la dimensionalidad tiene dos caminos. **Seleccionar** conserva un subconjunto de las variables originales (mantiene interpretabilidad). **Extraer** crea variables nuevas más compactas (gana compresión). Sabrás elegir el camino correcto según tu objetivo: explicar, comprimir o visualizar.

1 Objetivos de aprendizaje

Al finalizar la práctica, el estudiante será capaz de reducir la dimensionalidad de un conjunto de datos aplicando técnicas de selección y de extracción de características, y de justificar cuándo conviene cada una. En concreto:

- Distinguir **selección** (subconjunto de variables originales) de **extracción** (variables nuevas combinadas).
- Aplicar las tres familias de selección: **filtro** (**SelectKBest**, información mutua), **envoltura** (**RFE**) y **embebido** (importancia de árboles, Lasso L1).
- Implementar **PCA** con escalado previo, interpretar la varianza explicada y elegir el número de componentes con un *scree plot*.
- Visualizar datos de alta dimensión con **t-SNE** y **UMAP**, leyendo sus gráficos con las precauciones correctas.
- Integrar todo en un **Pipeline** de scikit-learn y comparar estrategias contra un clasificador real.

2 Competencias que desarrolla

- Preprocesa y transforma datos para el modelado evitando fugas de información (*data leakage*), ajustando siempre con los datos de entrenamiento.

- Toma decisiones informadas sobre reducción de dimensionalidad según el objetivo (interpretar vs. comprimir vs. visualizar).
- Usa scikit-learn de forma idiomática: selectores, transformadores y pipelines encadenados.

3 Requisitos previos

- Python, NumPy y Pandas a nivel básico.
- Nociones de partición entrenamiento/prueba y de un clasificador simple (regresión logística).
- Haber cubierto el escalado de características ([StandardScaler](#)) en la guía previa.

4 Materiales y entorno

Antes de comenzar, verifica tu entorno y ejecuta la celda de configuración inicial. Trabajarás sobre el cuaderno base `seleccion-extraccion-caracteristicas.ipynb`.

Recurso	Detalle
Lenguaje	Python 3.x
Entorno	Jupyter Notebook / JupyterLab
Librerías base	numpy · pandas · matplotlib · scikit-learn
Opcional	umap-learn (para la sección de UMAP)
Cuaderno base	seleccion-extraccion-caracteristicas.ipynb

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
plt.rcParams["figure.figsize"] = (6, 4)
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
```

5 Marco teórico

MALDICIÓN DE LA DIMENSIONALIDAD

Al crecer el número de características, el espacio se vuelve tan vasto que los datos quedan dispersos: las distancias pierden significado y los modelos tienden a **sobreajustar** (memorizar ruido). Reducir características bien elegidas mejora velocidad, interpretabilidad y, a veces, exactitud.

► Seleccionar vs. extraer

Seleccionar es escoger un subconjunto de las características originales y descartar el resto: las mismas variables, pero menos. Extraer es crear variables nuevas —combinaciones de las originales— que condensan la información en menos dimensiones. Para seleccionar existen tres familias:

Familia	Cómo decide	Balance
Filtro	Estadística por característica, sin modelo	Muy rápido; ignora interacciones
Envoltura	Prueba subconjuntos con un modelo real	Preciso; costoso
Embebido	La selección ocurre durante el entrenamiento	Buen término medio

6 Desarrollo del laboratorio

► 6.0 · Preparación de los datos

Usaremos dos conjuntos de datos con propósitos distintos.

DATASET A – SINTÉTICO, PARA SELECCIÓN

Lo fabricamos con `make_classification` sabiendo de antemano qué columnas sirven, así podremos **verificar** si cada método recupera las correctas: `f0–f3` informativas, `f4–f5` redundantes, `f6–f9` ruido.

In [2]:

```
from sklearn.datasets import make_classification

X_syn, y_syn = make_classification(
    n_samples=400, n_features=10,
    n_informative=4, n_redundant=2, n_repeated=0,
    n_classes=2, shuffle=False, random_state=42)

cols = [f"f{i}" for i in range(10)]
X_syn_df = pd.DataFrame(X_syn, columns=cols)
VERDAD = {"informativas": [0,1,2,3], "redundantes": [4,5], "ruido": [6,7,8,9]}
```

El clásico Iris (4 características, 3 especies) servirá para la extracción y la visualización: es perfecto para proyectar de 4D a 2D.

In [3]:

```
from sklearn.datasets import load_iris

iris = load_iris()
X_iris, y_iris = iris.data, iris.target
```

► 6.1 · Selección de características

Filtro (filter)

Evalúan cada característica con una medida estadística independiente del modelo: un primer cribado muy rápido. Primero descartamos columnas casi constantes (varianza nula), luego nos quedamos con las *k* mejores según su relación con el objetivo mediante la prueba F de ANOVA.

In [4]:

```
from sklearn.feature_selection import VarianceThreshold, SelectKBest, f_classif

vt = VarianceThreshold(threshold=0.0)      # elimina columnas constantes
X_vt = vt.fit_transform(X_syn)

sel = SelectKBest(score_func=f_classif, k=4)
X_kbest = sel.fit_transform(X_syn, y_syn)
elegidas = np.where(sel.get_support())[0]
```

LO QUE ACABA DE PASAR

El filtro **no** recupera exactamente `f0–f3`. El test F mide cada columna **por separado**, y algunas features solo son informativas **en combinación** con otras. Esa es la gran limitación del filtro: ignora las interacciones.

NOTA

Para variables **categorías** se usa `chi2`; para relaciones **no lineales**, `mutual_info_classif` (información mutua), que capta dependencias que la correlación lineal no ve.

Envoltura (wrapper)

Usan un modelo real para evaluar subconjuntos de características: más precisos, más costosos. El clásico es RFE (Recursive Feature Elimination): entrena, descarta la menos importante y repite.

In [5]:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

rfe = RFE(estimator=LogisticRegression(max_iter=1000),
          n_features_to_select=4)
rfe.fit(X_syn, y_syn)
elegidas_rfe = np.where(rfe.support_)[0]
```

EL CONTRASTE CLAVE

RFE recupera **exactamente** `f0-f3`, ¡las 4 informativas! Donde el filtro univariado fallaba, el envoltorio acierta porque evalúa las columnas **juntas** con un modelo real. Ese es el compromiso: más cómputo a cambio de captar interacciones.

Embebido (embedded)

La selección ocurre durante el entrenamiento. Dos casos clásicos: la importancia de características de los árboles (con `SelectFromModel`) y la regularización Lasso (L1), que empuja a cero los coeficientes de las variables inútiles.

In [6]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

rf = RandomForestClassifier(random_state=42).fit(X_syn, y_syn)
sel_rf = SelectFromModel(rf, threshold="median", prefit=True)
X_rf = sel_rf.transform(X_syn)

# Lasso: los coeficientes en 0 quedan descartados
l1 = LogisticRegression(solver="saga", l1_ratio=1, C=0.1, max_iter=5000)
l1.fit(X_syn, y_syn)
sobreviven = np.where(np.abs(l1.coef_.ravel()) > 1e-6)[0]
```

Familia	Método usado	¿Recuperó las informativas?
Filtro	<code>SelectKBest(f_classif)</code>	Parcial — ignora interacciones
Envoltura	<code>RFE(LogisticRegression)</code>	Sí, exacto
Embebido	<code>RandomForest</code> / <code>Lasso L1</code>	Sí, con algo de ruido residual

CÓMO ELEGIR

Si necesitas **explicar** qué variables influyen, prefiere **selección** —sabes qué columnas quedaron—. Si solo te importa comprimir o visualizar, la **extracción** (siguiente sección) suele ser más potente.

► 6.2 · Extracción de características: PCA

Cuando en vez de descartar columnas queremos condensarlas, usamos extracción. El Análisis de Componentes Principales (PCA) crea variables nuevas —los componentes principales— como combinaciones de las originales, ordenadas para que las primeras capturen la mayor parte de la variación.

PCA

Rota los ejes del espacio de datos para alinearlos con las direcciones de **máxima varianza**. El primer componente es la dirección donde los datos más se dispersan; el segundo, la siguiente (perpendicular al primero); etc. Conservando solo los primeros, reducimos dimensiones perdiendo la menor información posible.

ESCALAR ES OBLIGATORIO

PCA se basa en la varianza, y la varianza depende de las unidades. Si una columna está en miles (salario) y otra en decenas (edad), PCA creará que la primera es más importante solo por su escala. Aplica siempre **StandardScaler** **antes** de PCA.

In [7]:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_esc = StandardScaler().fit_transform(X_iris) # 1) escalar primero
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_esc) # 2) 4 columnas -> 2 componentes
evr = pca.explained_variance_ratio_
```

VARIANZA EXPLICADA (EXPLAINED_VARIANCE_RATIO_)

Proporción de la variación total que captura cada componente. Es la brújula para decidir cuántos conservar: en Iris, **2 de 4** componentes retienen ~96% de la información, y las tres especies se separan con notable claridad pese a pasar de 4D a 2D.

Para decidir cuántos componentes conservar, se mira la varianza acumulada y se elige el umbral (p. ej. 95%). El scree plot lo hace evidente:

In [8]:

```
pca_full = PCA().fit(X_esc)
acum = np.cumsum(pca_full.explained_variance_ratio_)

x = np.arange(1, len(acum) + 1)
fig, ax = plt.subplots(figsize=(6.5, 4))
ax.bar(x, pca_full.explained_variance_ratio_, color="#4F46E5", alpha=0.7)
ax.plot(x, acum, "o-", color="#F43F5E", label="acumulada")
ax.axhline(0.95, ls="--", color="gray", lw=1, label="umbral 95%")
plt.show()
```

EL PRECIO DE PCA

Los componentes son combinaciones matemáticas de las variables originales, así que pierden significado directo: *Componente 1* no es una magnitud del mundo real como *ancho de pétalo*. Ganamos compresión a cambio de interpretabilidad —el compromiso central de la extracción—.

► 6.3 · Reducción para visualización: t-SNE y UMAP

PCA es lineal. Cuando la estructura de los datos es compleja y curva, hay métodos no lineales pensados para ver datos de alta dimensión en 2D, revelando agrupamientos que PCA podría no mostrar.

CÓMO (NO) LEER UN GRÁFICO T-SNE

- Solo sirve para **visualizar**: no se usa como entrada de un modelo.
- El tamaño de los grupos y las distancias entre ellos **no** son literales; solo importa qué puntos se agrupan con cuáles.
- Es **estocástico**: cada corrida difiere. Fija `random_state` y prueba varios `perplexity`.

In [9]:

```
from sklearn.manifold import TSNE

X_tsne = TSNE(n_components=2, perplexity=30,
              random_state=42).fit_transform(X_esc)

# UMAP (requiere: pip install umap-learn) – similar objetivo,
# suele ser más rápido y preserva mejor la estructura global:
# import umap
# X_umap = umap.UMAP(n_components=2, random_state=42).fit_transform(X_esc)
```

	PCA	t-SNE / UMAP
Tipo	Lineal	No lineal
Uso principal	Comprimir para modelar o visualizar	Visualizar agrupamientos
¿Alimentar un modelo?	Sí, es habitual	No; solo exploración visual
Determinista	Sí	No (depende de la semilla)

► 6.4 · Síntesis: un mini-pipeline que junta todo

LA REGLA DE FONDO

Igual que en todo el preprocesamiento: **aprende los parámetros solo del entrenamiento** y aplica la misma transformación al resto. Por eso todo va dentro de un `Pipeline` y se ajusta con los datos de `train`.

In [10]:

```

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline

base = make_pipeline(StandardScaler(),
                    LogisticRegression(max_iter=5000))
seleccion = make_pipeline(StandardScaler(),
                        SelectKBest(f_classif, k=10),
                        LogisticRegression(max_iter=5000))
extraccion = make_pipeline(StandardScaler(),
                          PCA(n_components=0.95),
                          LogisticRegression(max_iter=5000))

```

Moraleja: reducir de 30 columnas a ~10 (o a los componentes que retienen el 95% de la varianza) casi no cuesta exactitud —y a veces la mejora—, ganando velocidad e interpretabilidad. Eso es exactamente lo que buscábamos.

7 Entregables

- El cuaderno `seleccion-extraccion-caracteristicas.ipynb` con **todas** las celdas de ejercicio resueltas y ejecutadas (con sus salidas visibles).
- Una breve reflexión (5–8 líneas) al final del cuaderno: ¿cuándo elegirías selección y cuándo extracción? Justifica con lo observado en el ejercicio final.
- Nombre del archivo: `GL08_Apellido_Nombre.ipynb`.

9 Rúbrica de evaluación

Criterio	Descripción	Pts.
Selección	Las tres familias implementadas y comentadas correctamente	30
Extracción (PCA)	Escalado previo, varianza explicada y scree plot interpretados	25
Visualización	t-SNE (y/o UMAP) generado y leído con las precauciones correctas	15
Pipeline final	Comparación de estrategias sin fuga de datos, con conclusión	20
Reflexión y forma	Análisis claro; cuaderno ordenado y ejecutado de principio a fin	10
Total		100

10 Referencias

- scikit-learn — *Feature selection y Decomposition (PCA)*, scikit-learn.org/stable/modules/feature_selection.html.
- scikit-learn — *Manifold learning (t-SNE)*; UMAP — umap-learn.readthedocs.io.
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (cap. sobre reducción de dimensionalidad).
- James, Witten, Hastie & Tibshirani. *An Introduction to Statistical Learning* (PCA y selección de subconjuntos).

FIN DE LA GUÍA

Repasaste las tres familias de selección (filtro, envoltura, embebido), la extracción con PCA (escalar → varianza explicada → scree plot) y la visualización no lineal con t-SNE y UMAP, cerrando con un pipeline comparativo. INF672 · Unidad 2 · 2.4 – M. Sc. Huáscar F. Gonzales G.